

Linux Server Operations Lab

Phase 7 レビュー - Ansibleの点検結果をExcelへ自動書き戻し

今回の到達点

Excelのサーバ管理台帳を起点に、AnsibleでLinux 3台の状態を取得し、Pythonで判定した結果を同じExcelブックへ自動的に書き戻せるところまで完成した。さらにVirtualBox共有フォルダを導入し、Macとcontroller間でExcelファイルをscp往復させる必要もなくなった。

01_SERVER_MASTER	Python	Ansible	Linux 3台	02_CHECK_RESULTS
サーバ情報	読み込み・判定	状態取得	tokyo / yokohama / omiya	結果を書き戻し

Phase 7 完了条件

3台分のDisk使用率・nginx状態・総合Result・実行日時がExcelの「02_CHECK_RESULTS」へ自動反映され、ServerID順に並び、Resultが色分けされること。

1. 構成と役割

要素	役割
Mac / Excel	台帳の編集・結果確認。Linux_Server_Operations_Lab.xlsx を管理する。
VirtualBox共有フォルダ	Macの Ansible_Python フォルダをcontrollerから直接参照できるようにする。
controller	PythonとAnsibleを実行する制御サーバ。
Ansible	tokyo / yokohama / omiya に接続してDiskとnginx状態を取得する。
Python	Ansible出力を解析し、OK / WARNING / FAILED / UNREACHABLEを判定してExcelへ書き込む。
01_SERVER_MASTER	ServerID、Hostname、IP、OS、Role、Environmentの正本。
02_CHECK_RESULTS	点検結果の出力先。ServerID、Hostname、Disk、nginx、Result、ExecutedAtを保持する。

共有フォルダの最終状態

```
/media/sf_Ansible_Python
└── Linux_Server_Operations_Lab.xlsx
```

この構成により、Macで保存したExcelをcontrollerのPythonが同じファイルとして読み書きできる。scpで「Mac -> controller -> Mac」と往復させる必要がなくなった。

2. 今回使用した主な処理

Disk使用率の取得

```
ansible servers -i hosts_generated.ini -a "df -P /"
```

PythonではAnsibleの標準出力から、ルートファイルシステムの使用率を正規表現で取り出す。80%以上ならWARNING、それ未満ならOKと判定する。

```
disk_match = re.search(r"(\d+)\% \s+/$", line)
```

```
if disk_percent >= 80:
    status = "WARNING"
else:
    status = "OK"
```

nginx状態の取得

```
ansible servers -i hosts_generated.ini -a "systemctl is-active nginx"
```

activeならOK、接続不能ならUNREACHABLE、コマンド失敗などはFAILEDとして扱う。Diskとnginxの結果を統合して最終Resultを決める。

```
UNREACHABLE が1つでもある -> UNREACHABLE
FAILED が1つでもある      -> FAILED
Diskが80%以上             -> WARNING
それ以外                  -> OK
```

ServerIDの取得

SV001などをPythonに直接ハードコードせず、01_SERVER_MASTERからHostnameとServerIDの対応を読み取る。これにより、Excelをサーバ情報の正本として扱える。

```
server_ids["tokyo"]  -> "SV001"
server_ids["yokohama"] -> "SV002"
server_ids["omiya"]  -> "SV003"
```

3. Excel書き戻し結果

✔ **ブックが修復されました** このブックは一時的に修復されたため、閲覧ビューで開くことができます。 [フィードバックの送信](#)

詳細を表示... ブ

	A	B	C	D	E	F
1	ServerID	Hostname	Disk	nginx	Result	ExecutedAt
2	SV001	tokyo	25%	OK	OK	2026-09-21 01:44:29
3	SV002	yokohama	16%	OK	OK	2026-09-21 01:44:29
4	SV003	omiya	16%	OK	OK	2026-09-21 01:44:29
5						

最終結果

ServerID	Hostname	Disk	nginx	Result
SV001	tokyo	25%	OK	OK
SV002	yokohama	16%	OK	OK
SV003	omiya	16%	OK	OK

見た目の仕上げとして、ServerID順に並べ、ヘッダーを強調し、Result=OKを緑、WARNINGを黄、FAILED/UNREACHABLEを赤にする処理も追加した。ExecutedAtには実行日時を自動記録している。

4. トラブルシューティング記録

事象	原因	切り分け・対応
02_CHECK_RESULTS does not exist	controller側のExcelが古く、Macで追加したシーマの最新版Excelをcontrollerへ送る必要があると判明。	
Macから /home/controller/... を開こうとMac File not found Error	MacのファイルシステムとLinux側のファイルシステムを混同した。プロンプトを見て、Macかcontrollerかを常に確認する。	
共有フォルダが Permission denied	VirtualBox共有フォルダが root:root でマウントされていず、見えないことを確認し、Linux側の権限問題と切り分けた。	
3台すべてUNREACHABLE	tokyo / yokohama / omiya のVMが停止していた。Ansible pingで疎通確認後、3台を起動して再実行。	

今回の学び

エラーを見たときに、すぐコードを疑うのではなく「どのマシンで実行しているか」「どのファイルを参照しているか」「共有自体は成功しているか」「権限だけが問題か」と順番に切り分けることが重要だった。特に `sudo ls` で中身が見えたことで、VirtualBox共有そのものではなくLinux側の権限問題と判断できた。

5. Phase 7で得た実務的な意味

今回の成果は単なるExcel自動入力ではない。運用台帳に登録したサーバ情報を元に、Linuxの実状態をAnsibleで確認し、その結果を自動で台帳へ戻す仕組みになっている。手作業で各サーバへSSHし、dfやsystemctlを実行してExcelへ転記する作業を、1つのPythonスクリプトにまとめられた。

手作業	今回の自動化
各サーバへSSH	Ansibleで3台を一括確認
df結果を目視	PythonでDisk使用率を抽出
nginx状態を目視	Pythonでactive/失敗/接続不能を判定
Excelへ手入力	openpyxlで自動書き戻し
異常判断を人が行う	OK/WARNING/FAILED/UNREACHABLEを自動分類

note記事候補

「Excelをサーバ運用台帳にして、Ansibleの点検結果をPythonで自動書き戻ししてみた」

「VirtualBox共有フォルダでMacとLinuxのExcelを1つにした」

「UNREACHABLEもExcelへ自動反映する運用チェックを作る」

次のPhase

Phase 8では、ExcelからPythonへ作業指示を渡し、その内容をAnsible経由でLinuxへ反映する。Phase 7が「Linux -> Excel」の流れだったのに対し、Phase 8では「Excel -> Linux」の逆方向を作り、Excelを台帳から操作盤へ発展させる。

Phase 7 完了

Excel × Python × Ansible × Linux の一連の連携が成立した。